

```

from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier, plot_tree, export_text
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
iris = load_iris()
X, y = iris.data, iris.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

```

```

tree = DecisionTreeClassifier(max_depth=2, random_state=42)
tree.fit(X_train, y_train)
print(f'Train accuracy: {tree.score(X_train, y_train):.2%}')
print(f'Test accuracy: {tree.score(X_test, y_test):.2%}')

```

Train accuracy: 95.00%  
Test accuracy: 96.67%

```
print(export_text(tree, feature_names=iris.feature_names))
```

```

|--- petal length (cm) <= 2.45
|   |--- class: 0
|--- petal length (cm) > 2.45
|   |--- petal length (cm) <= 4.75
|       |--- class: 1
|   |--- petal length (cm) > 4.75
|       |--- class: 2

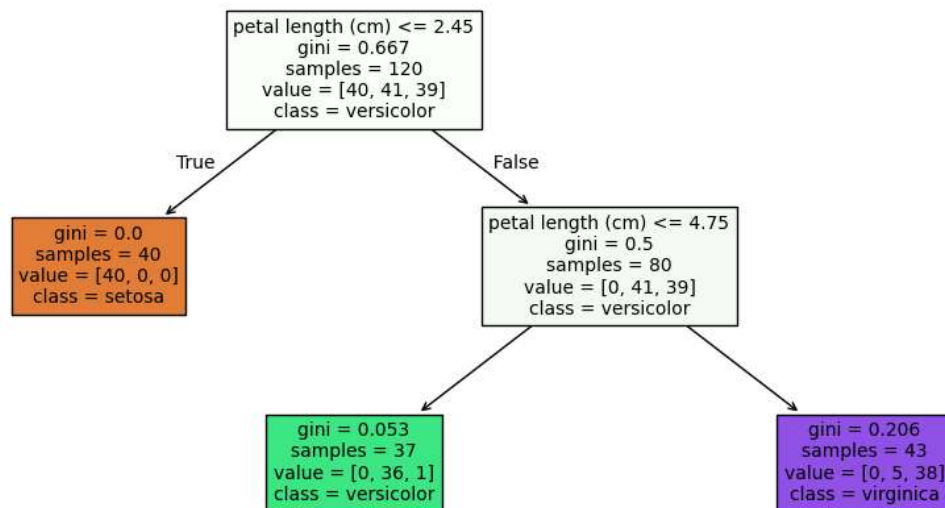
```

```

plt.figure(figsize=(10, 5))
plot_tree(tree, feature_names=iris.feature_names,
          class_names=iris.target_names,
          filled=True, rounded=False, fontsize=10)
plt.title('Iris Decision Tree (max_depth=2)'); plt.tight_layout(); plt.show()

```

Iris Decision Tree (max\_depth=2)



```

new_flower = [[5.1, 3.5, 1.4, 0.2]]
pred = tree.predict(new_flower)[0]
print(f'Predicted species: {iris.target_names[pred]}')
print(f'Probabilities : {tree.predict_proba(new_flower)[0]}')

```

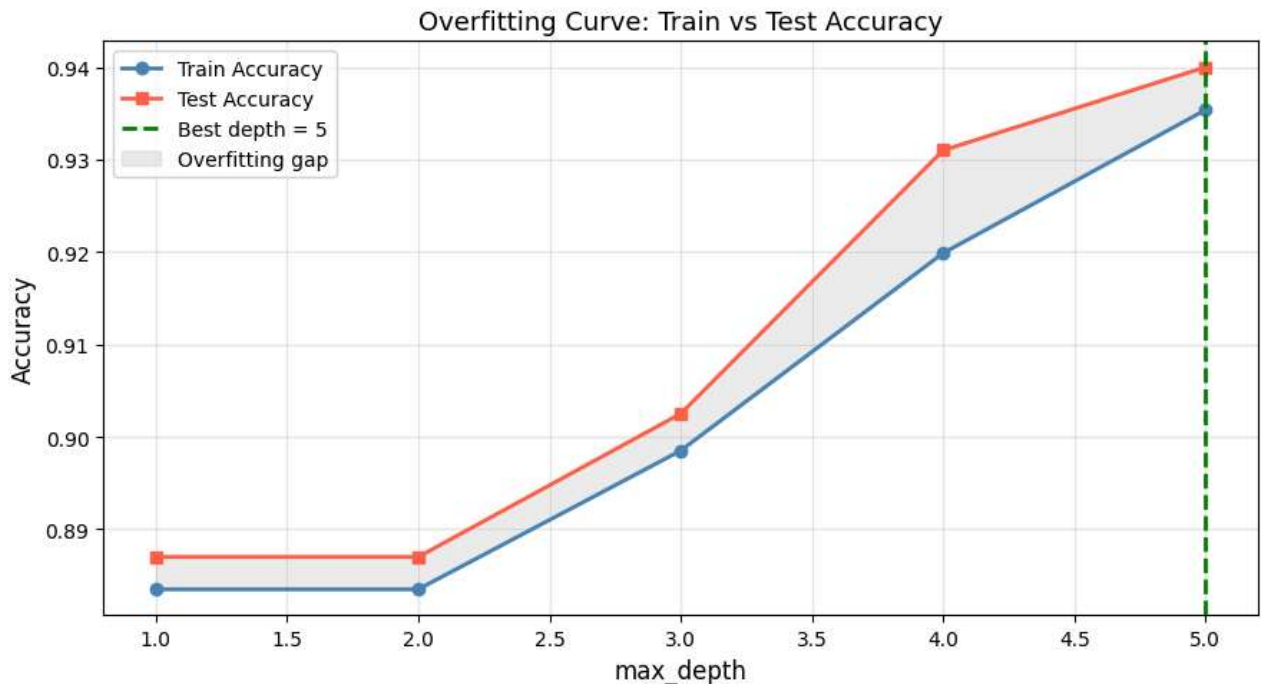
Predicted species: setosa  
Probabilities : [1. 0. 0.]

```

from sklearn.tree import DecisionTreeClassifier
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt

X, y = make_classification(n_samples=10000, n_features=10, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
depths=range(1,6)
train_acc,test_acc=[],[]
for d in depths:
    tree=DecisionTreeClassifier(max_depth=d,random_state=42)
    tree.fit(X_train, y_train)
    train_acc.append(accuracy_score(y_train, tree.predict(X_train)))
    test_acc.append(accuracy_score(y_test, tree.predict(X_test)))
best_depth = depths[test_acc.index(max(test_acc))]
plt.figure(figsize=(9, 5))
plt.plot(depths, train_acc, 'o-', color='steelblue', lw=2, label='Train Accuracy')
plt.plot(depths, test_acc, 's-', color='tomato', lw=2, label='Test Accuracy')
plt.axvline(x=best_depth, color='green', linestyle='--', lw=2,
            label=f'Best depth = {best_depth}')
plt.fill_between(depths,
                train_acc, test_acc,
                alpha=0.15, color='gray', label='Overfitting gap')
plt.xlabel('max_depth', fontsize=12)
plt.ylabel('Accuracy', fontsize=12)
plt.title('Overfitting Curve: Train vs Test Accuracy', fontsize=13)
plt.legend(); plt.grid(alpha=0.3); plt.tight_layout(); plt.show()
print(f'Best test accuracy {max(test_acc):.2%} at depth={best_depth}')

```



Best test accuracy 94.00% at depth=5

```

from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import cross_val_score
import numpy as np
from sklearn.datasets import make_classification

X, y = make_classification(n_samples=10000, n_features=10, random_state=42)

configs = [
    {'max_depth': None},
    {'max_depth': 5},
    {'max_depth': 5, 'min_samples_split': 10},
    {'max_depth': 5, 'min_samples_leaf': 5},
    {'max_depth': 5, 'max_features': 'sqrt'},
]

```

```
print(f'{'Config':45s} {'CV Score (mean±std)':20s}')
print('-' * 68)
for cfg in configs:
    tree = DecisionTreeClassifier(**cfg, random_state=42)
    scores = cross_val_score(tree, X, y, cv=5, scoring='accuracy')
    print(f'{'str(cfg):45s} {scores.mean():.3f} ± {scores.std():.3f}')
```

| Config                                    | CV Score (mean±std) |
|-------------------------------------------|---------------------|
| -----                                     | -----               |
| {'max_depth': None}                       | 0.900 ± 0.004       |
| {'max_depth': 5}                          | 0.926 ± 0.008       |
| {'max_depth': 5, 'min_samples_split': 10} | 0.926 ± 0.008       |
| {'max_depth': 5, 'min_samples_leaf': 5}   | 0.926 ± 0.008       |
| {'max_depth': 5, 'max_features': 'sqrt'}  | 0.917 ± 0.006       |